2026

Intelligent RAG-Driven Customer Support Ecosystem - HLD

DATA SCIENCE INTERNSHIP – FEBRUARY 2026

VEDIKA PUNJABI - IN226101802

Table of Contents

1. System Overview 2
   1.1 Problem Definition 2
   1.2 Scope 2
2. Architecture Walkthrough 2
   2.1 The Ingestion Pipeline (Offline Phase) 2
   2.2 The Query & Orchestration Flow (Online Phase) 3
3. Component Description 4
4. Data Flow 5
5. Technology Choices (The "Tech Stack") 5
6. Scalability Considerations 6

1. System Overview

1.1 Problem Definition

Support teams often struggle with high volumes of repetitive queries. While standard LLMs can help, they often provide inaccurate "hallucinations" regarding specific company policies. There is a need for a system that provides fact-based answers while allowing humans to oversee and correct the AI when the complexity exceeds its capabilities.

1.2 Scope

This system covers the automated processing of policy documents, intelligent retrieval of facts, and a state-managed workflow that integrates human expertise via a Human-in-the-Loop (HITL) mechanism.

2. Architecture Walkthrough

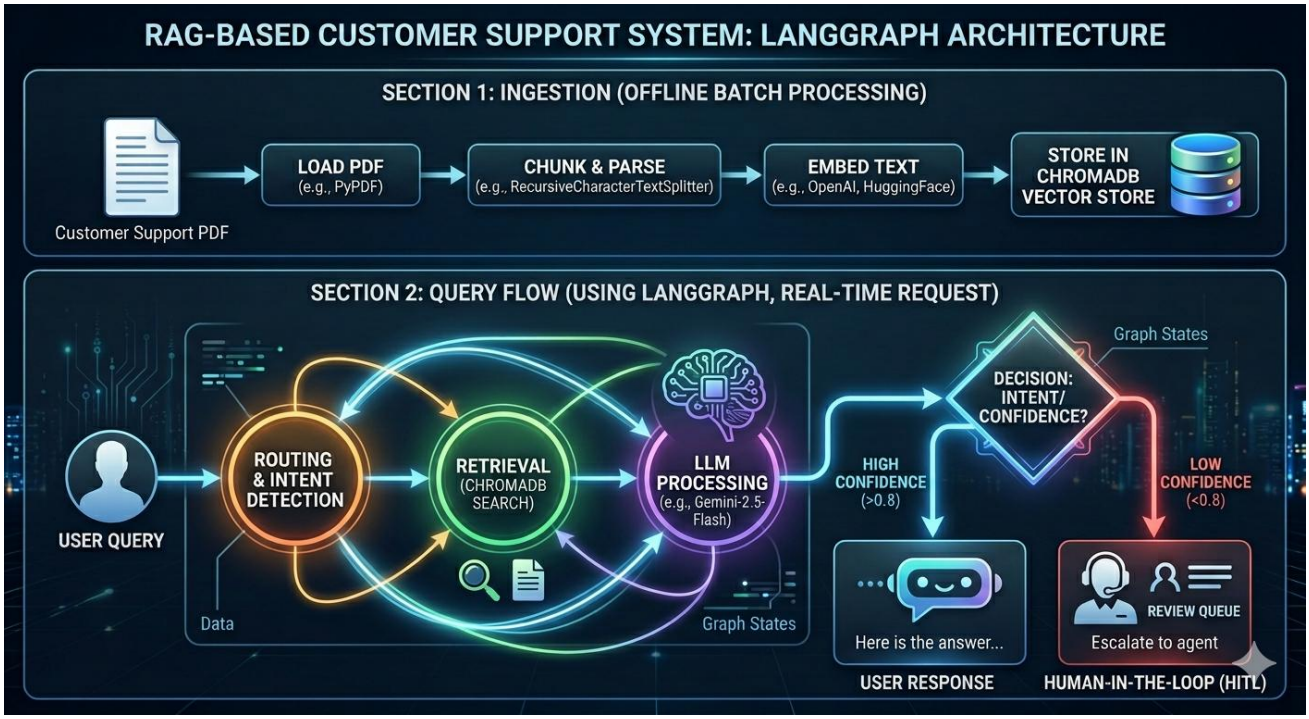

Figure 1: Dual-Phase RAG Architecture & LangGraph Workflow Logic

The system architecture is divided into two distinct logical phases to ensure modularity and scalability:

2.1 The Ingestion Pipeline (Offline Phase)

- **Data Extraction:** The system uses `PyPDFLoader` to parse unstructured data from the University of Arkansas policy documents.
- **Contextual Chunking:** To ensure the LLM receives meaningful context without exceeding token limits, the text is split into 1000-character segments with a 150-character overlap.

- **Vectorization:** Each chunk is converted into a 768-dimensional vector using the `gemini-embedding-001` model, capturing the semantic meaning of the text.
- **Storage:** These vectors are stored in a local `ChromaDB` instance, allowing for sub-second similarity searches during the query phase.

2.2 The Query & Orchestration Flow (Online Phase)

- **Semantic Retrieval:** When a user asks a question, the system converts the query into a vector and retrieves the top 3 most relevant chunks from ChromaDB.
- **Stateful Orchestration (LangGraph):** The entire process is managed as a directed graph. The "State" object carries the conversation history and retrieved context between nodes.
- **Decision-Making (Conditional Routing):** The **Assistant Node** uses **Gemini 2.5 Flash Lite** to analyse if the retrieved context is sufficient.

  - ✓ If **sufficient**, it generates a response grounded in the PDF data.

  - ✓ If **insufficient**, it triggers an "ESCALATE" signal, moving the graph to the **Human Review Node**.

- **Human-in-the-Loop (HITL):** This node utilizes the LangGraph `interrupt` feature, pausing execution until a human expert provides a verified answer, ensuring 100% accuracy for complex queries.

3. Component Description

Component	Detail	Responsibility
Document Loader	`PyPDFLoader`	Converts PDF pages into document objects for processing.
Chunking Strategy	`RecursiveCharacterTextSplitter`	Breaks text into 1000-character segments with 150-character overlap to preserve semantic context.
Embedding Model	`gemini-embedding-001`	Generates high-dimensional vector representations of text.
Vector Store	`ChromaDB`	Indexes and stores vectors for fast similarity-based retrieval.
Retriever Layer	Similarity Search ($k = 3$)	Identifies the top 3 most relevant document chunks for any user query.
LLM (Reasoning)	Gemini 2.5 Flash Lite	Processes retrieved context and uses "Thinking" to generate answers or trigger escalation.
Workflow Engine	`LangGraph`	Controls the execution flow, state management, and "interrupt" functionality.
HITL Module	`interrupt()` & `resume`	Pauses the graph to allow a human to provide a verified response.

4. Data Flow

- **Ingestion:** The PDF is loaded, chunked, and embedded into ChromaDB.
- **Input:** User submits a support query.
- **Processing:**
  - ✓ The Retriever fetches the most relevant context from ChromaDB.
  - ✓ The LLM analyses the query + context.
- **Conditional Routing:**
  - ✓ **Case A:** If the LLM has enough information, it generates a response.
  - ✓ **Case B (Escalation):** If context is missing, the graph "interrupts" and requests human input.
- **Output:** The user receives the finalized answer (either AI-generated or human-verified).

5. Technology Choices (The "Tech Stack")

Component	Selection	Justification
Orchestration	***LangGraph***	Enables stateful, multi-turn conversations and native "Interrupt" capability for HITL.
LLM / Reasoning	***Gemini 2.5 Flash Lite***	Chosen for its high-speed performance and native "Thinking" capability for better routing decisions.
Embeddings	***gemini-embedding-001***	Provides high-dimensional semantic understanding optimized for the Gemini ecosystem.
Vector DB	***ChromaDB***	An open-source, light-weight vector database that allows for easy local persistence and fast similarity search.
Language	***Python 3.10+***	The standard for AI development with robust library support for LangChain and Google Generative AI.

6. Scalability Considerations

To transition this from a Colab prototype to a production-grade system, the following design considerations are included:

- **Vector DB Scalability:** For larger datasets, ChromaDB can be transitioned to a distributed service (like Pinecone or Weaviate) without changing the core graph logic.
- **Stateless Inference:** The use of **LangGraph Checkpointers** (MemorySaver) allows the system to handle thousands of concurrent "threads" (conversations) by saving their states to a persistent database.
- **Model Rate Limiting:** The design accounts for API rate limits by using Gemini 2.5 Flash Lite, which offers higher throughput than larger "Pro" models.
- **Asynchronous Processing:** The nodes are designed to support `async` execution, allowing the system to handle multiple document retrievals simultaneously.